

A Supplemental Material: Extended SEN Angle

The extended SEN angle bound is derived by assuming d neighbors with pairwise constant angle. To obtain that, we first define the non-normalized neighbors as $n'_i := \mathbf{1} + ae_i$ where $\mathbf{1}$ is the all-ones vector, e_i is the i -th unit vector, and a is a constant to control for the pairwise angles. The effective neighbors used are then the normalized vectors $n_i = \frac{n'_i}{\|n'_i\|}$. By construction

$$(n_i)_j = \begin{cases} \frac{1}{\sqrt{d-1+(1+a)^2}} & \text{if } i \neq j \\ \frac{1+a}{\sqrt{d-1+(1+a)^2}} & \text{if } i = j \end{cases} \quad (1)$$

Thereby the cosine of the angle between any two neighbors n_i and n_j with $i \neq j$ is given by

$$\langle n_i, n_j \rangle = \frac{2a + d}{d - 1 + (1 + a)^2} \quad (2)$$

The spherical center c of these neighbors is the normalized all-ones vector, i.e. the vector consisting of all $1/\sqrt{d}$. The angle between all neighbors and the spherical center is thus

$$\langle n_i, c \rangle = \frac{a + d}{\sqrt{d(d - 1 + (1 + a)^2)}} \quad (3)$$

By solving the second equation for a , given a fixed value for $\langle n_i, c \rangle$ and then substituting the solution into the first equation, we obtain the angle bounds given in the paper. Note that the second equation gives two solutions for a , one positive and one negative. The positive solution is the one we are interested in, as for that parameterization, all neighbors lie on the same hemisphere as the all-ones vector. Both solutions for a , however, lead to the same value of $\langle n_i, n_j \rangle$, as the second solution gives n_i mirrored at the normal plane to $\mathbf{1}$, which changes neither the cosine of the angle to the center nor the cosine of any pairwise angle between neighbors.

B Supplemental Material: Experiments

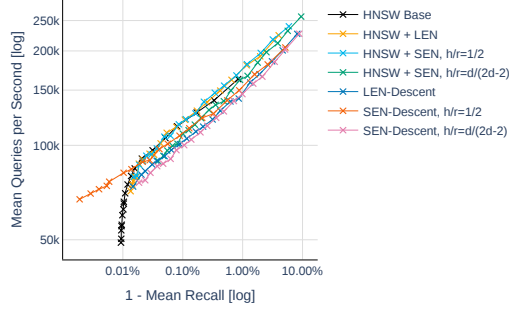
All experiments were conducted on a single computer with a 10-core (physical, 20-core logical) Intel Xeon W-2155 CPU and 64GB of RAM. All reported times are wall-clock times, measured with the `time` Python package. The code was written in Rust and controlled via a Python interface. For our experiments, we considered an HNSW implementation with varying heuristics and a RNN-Descent style implementation using either the lens or sector exclusion criterion. We call these variants LEN-Descent (essentially RNN-Descent) and SEN-Descent (essentially NSSG with configurable sector heights). For the Descent algorithms, we always use random initialization, 4 outer loops, and 30 inner loops. We implemented HNSW to not use an exclusion criterion during construction but

approximate the kNNG. To that end, we store each adjacency list during construction in a max heap, such that the most distant neighbor can be dropped when the out-degree exceeds the desired value. At the end, we can optionally prune to the LENs or SENs. The upper-bound for the out-degree is fixed to 50 in the bottom layer (M in HNSW nomenclature) and 25 for any higher layer. We used a constant nearest neighbor candidate count (`ef_construct` in HNSW nomenclature) of 200 during construction of all HNSW variants. All graphs are converted to an unweighted directed list-of-list-style graph structure for searching. The graphs then share the same searcher code base with either a capped or uncapped search queue. If not specified, we assume the uncapped search, as it is the default in the literature. For single-graph indexes, we can optionally set the entry points to a custom list. All queries were evaluated for the $10@n$ -recall, where n (`ef` in HNSW nomenclature) was varied from 10 to 100 to produce curves. As datasets we used uniform random ball distributions in different dimensions to roughly comply with the Poisson Point Process model. We further used the GooAQ dataset provided by this years SISAP indexing challenge¹ as a “real world” dataset example. The GooAQ dataset consists of roughly 3 million embeddings of google search queries with 384 dimensions. For each test, we used 1 million samples at random as dataset and 10 000 random samples as queries. Each experiment was run 10 times of varying random samples and the results were aggregated.

B.1 Pruning Heuristics

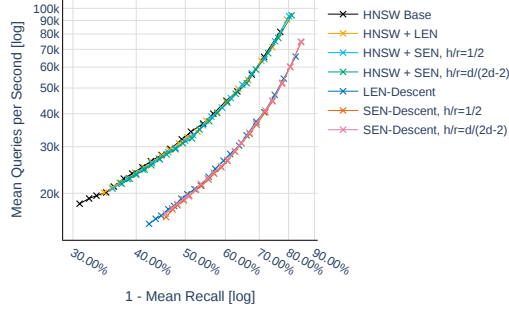
The first comparison we want to provide is between unpruned HNSW, HNSW with LEN/SEN pruning, and LEN/SEN-Descent. The “QPS over recall” curves, construction times, and out-degrees (i.e. graph sizes) are displayed in Fig. 1. In lower dimensions, the differences between the methods are almost negligible, with HNSW performing slightly better than the descent methods. In higher dimensions, however, the differences between HNSW and the descent methods become more pronounced. The SEN-Descent variants perform better than LEN-Descent, and the increased angle achieves a smaller graph size with comparable recall to the 60° bound. The HNSW variants do not vary in performance on the synthetic datasets but on GooAQ, using either pruning heuristic is better. The out-degrees vary, for one indicating that the insertion construction failed to approximate the kNNG, for the other again showing that the increased angle bound for SENs achieves the smallest graph with comparable recall. Surprisingly, the HNSW with LEN pruning out-degrees were larger than with SEN pruning. We assume that to be due to the bad kNNG approximation. Surprisingly, the descent variants performed best on the GooAQ dataset, which might be due to a low intrinsic dimensionality of these datasets or distributional effects.

10 dimensional uniform ball



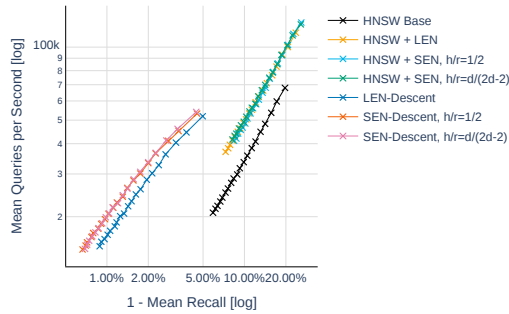
Method	Build time	Out-degree
HNSW Base	25.61 $\pm$ 0.16s	49.99 $\pm$ 0.49
HNSW + LEN	26.22 $\pm$ 0.12s	24.15 $\pm$ 3.00
HNSW + SEN, $h/r=1/2$	26.19 $\pm$ 0.28s	18.79 $\pm$ 2.78
HNSW + SEN, $h/r=d/(2d-2)$	26.03 $\pm$ 0.15s	15.11 $\pm$ 2.36
LEN-Descent	24.30 $\pm$ 0.05s	17.31 $\pm$ 5.13
SEN-Descent, $h/r=1/2$	19.94 $\pm$ 0.07s	22.15 $\pm$ 5.00
SEN-Descent, $h/r=d/(2d-2)$	20.29 $\pm$ 0.05s	17.35 $\pm$ 4.01

50 dimensional uniform ball



Method	Build time	Out-degree
HNSW Base	78.17 $\pm$ 0.15s	49.99 $\pm$ 0.51
HNSW + LEN	80.00 $\pm$ 0.15s	42.98 $\pm$ 2.76
HNSW + SEN, $h/r=1/2$	79.97 $\pm$ 0.24s	40.80 $\pm$ 3.14
HNSW + SEN, $h/r=d/(2d-2)$	80.05 $\pm$ 0.27s	38.83 $\pm$ 3.35
LEN-Descent	28.92 $\pm$ 0.10s	44.76 $\pm$ 16.37
SEN-Descent, $h/r=1/2$	26.07 $\pm$ 0.09s	44.96 $\pm$ 10.20
SEN-Descent, $h/r=d/(2d-2)$	26.35 $\pm$ 0.11s	43.83 $\pm$ 11.05

GooAQ sample



Method	Build time	Out-degree
HNSW Base	93.26 $\pm$ 0.29s	49.99 $\pm$ 0.50
HNSW + LEN	95.15 $\pm$ 0.25s	14.90 $\pm$ 5.26
HNSW + SEN, $h/r=1/2$	95.00 $\pm$ 0.54s	11.49 $\pm$ 4.75
HNSW + SEN, $h/r=d/(2d-2)$	94.67 $\pm$ 0.29s	11.41 $\pm$ 4.72
LEN-Descent	49.26 $\pm$ 2.17s	22.32 $\pm$ 16.77
SEN-Descent, $h/r=1/2$	45.31 $\pm$ 0.56s	27.27 $\pm$ 6.26
SEN-Descent, $h/r=d/(2d-2)$	45.29 $\pm$ 0.56s	27.08 $\pm$ 6.26

Fig. 1. Comparison of HNSW with and without pruning, LEN-Descent, and SEN-Descent. The curves show the QPS over error (1-recall) for each method. The further up or left, the better. The corresponding table displays the construction times and out-degrees of the bottom layer for each method. The values in the table are given as “mean $\pm$ standard deviation”.

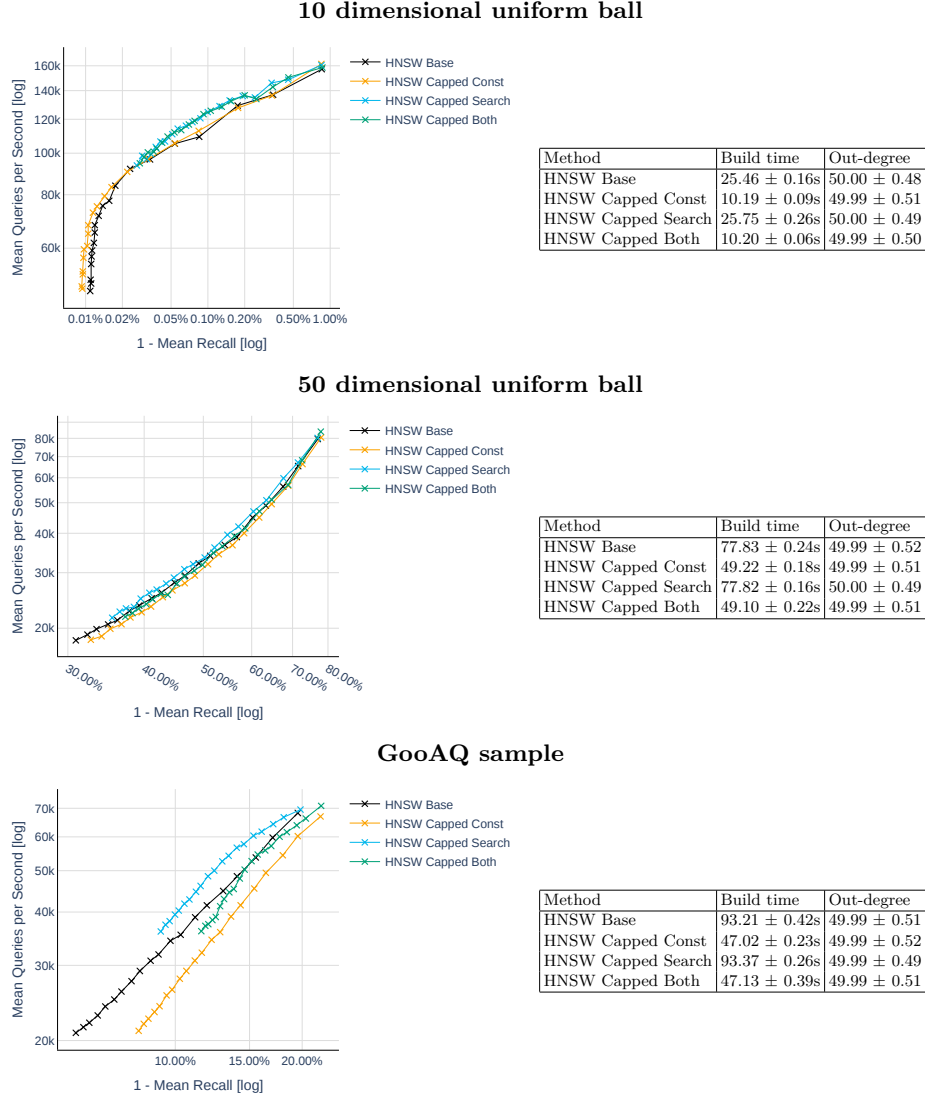


Fig. 2. Comparison of HNSW with and without capped search. The curves show the QPS over error (1-recall) for each method. The further up or left, the better. The corresponding table displays the construction times and out-degrees of the bottom layer for each method. The values in the table are given as “mean $\pm$ standard deviation”.

B.2 Capped Search

Next we compare the capped and uncapped search for construction and search. Both search routines during construction and query time are optionally capped to a maximum of 10 vertices. The results are displayed in Fig. 2. Using capped search during construction is significantly faster (up to 3x speedup) and does not degrade the query time recall a lot on synthetic data. On GooAQ, the capped construction slightly degrades the recall noticeably, but that might be a tradeoff worth taking for the speedup. The capped query search does not change the performance a lot on synthetic data, but can improve the query performance on GooAQ. It generally costs recall, but the speedup more than compensates for that. The capped search is thus a viable heuristic for construction and potentially for queries, dependent on the dataset.

B.3 Layer Construction

In the next experiment, we compare the default HNSW construction with a single-layer HNSW and a bottom-up vertex selection scheme for higher layers, which we call “flood-selection”. The flood-selection greedily selects all vertices for the next higher layer that cannot be reached by already selected vertices. If the navigation in each layer provides the nearest neighbor, the flooding technique should induce only a single step per layer. Each layer is for now constructed in isolation, although the heuristic can certainly be extended to a joint construction of all layers. The results are displayed in Fig. 3. In lower dimensions, the flooding technique performs best, although only slightly better than the standard and single-layer HNSWs. The difference vanishes in higher dimensions and in the presence of larger errors. On the GooAQ samples, the standard HNSW performed best. We do not have a clear intuition as to why GooAQ performs so differently, but we assume that the distribution of the dataset play a significant role. Ultimately, while the hierarchy is not useful in dense and high-dimensional data, it can be useful on certain distributions. The flooding technique is for now not useful, but might be extended to a viable heuristic in the future, when the navigation success per layer can be controlled more closely.

B.4 Randomization

The last experiments is about random edges and random entry points. We optionally add 3 random out-edges to each vertex (theoretically sufficient for percolation) or select 10 random vertices as entry points to the beam search. Both heuristics are designed to reduce the average path length in the graph. The standard HNSW construction induces a unique entry point from which the insertion position of a new vertex is determined. This entry point is usually used as the entry point to the query search as well. By randomizing the entry points and astarting at multiple positions, we want to raise the chance of starting the navigation closer to the query. As the entry points of the bottom layer are induced

¹ <https://sisap-challenges.github.io/2025/index.html>

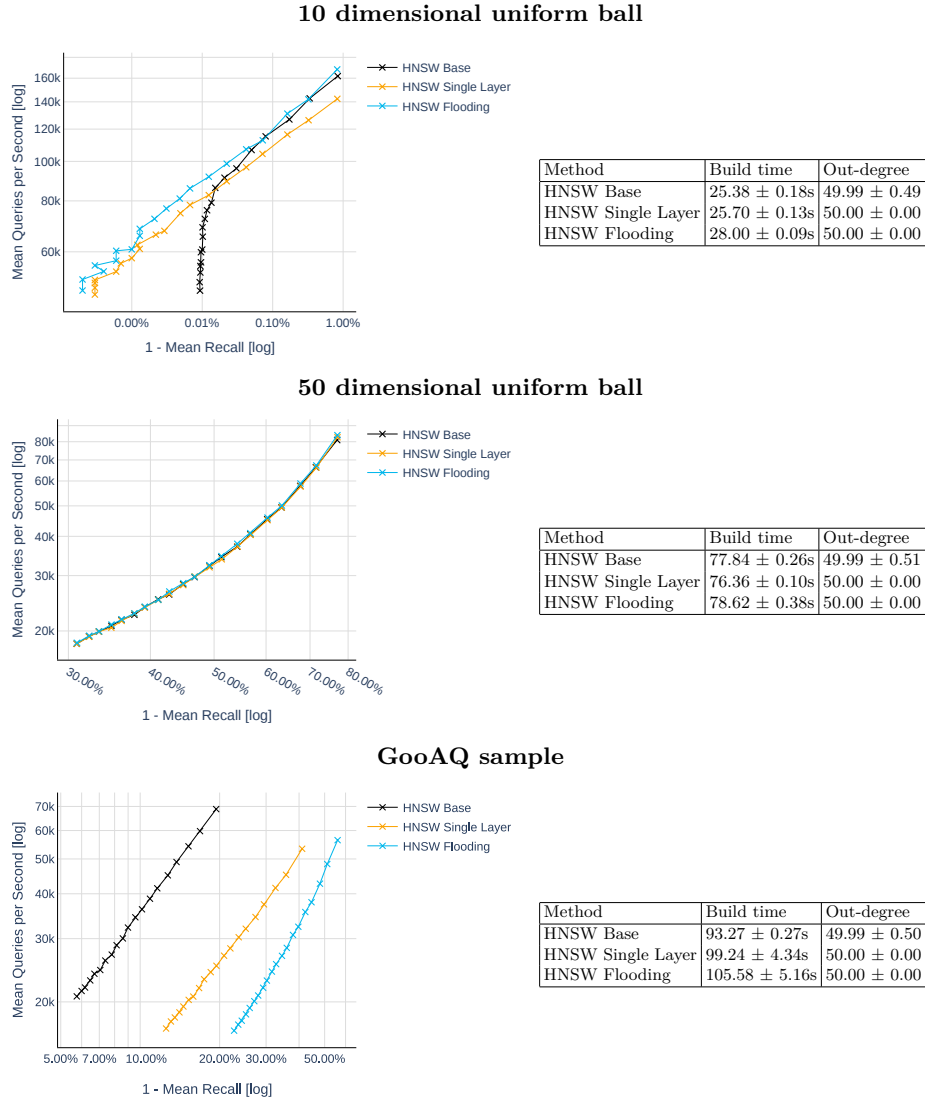
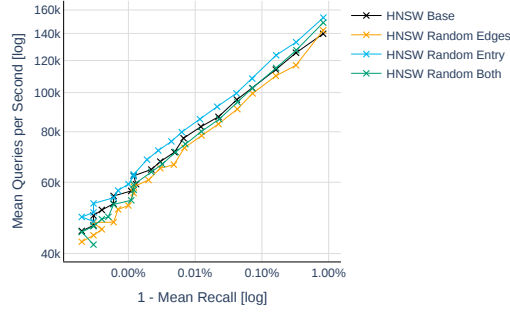


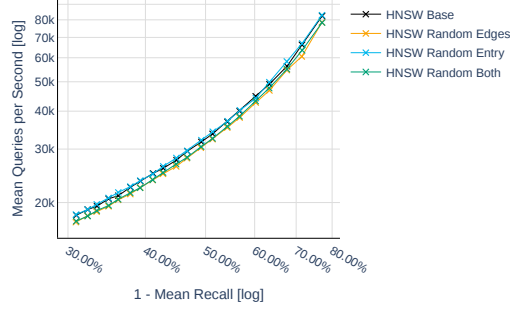
Fig. 3. Comparison of HNSW with default construction, a single layer, and flood-selection. The curves show the QPS over error (1-recall) for each method. The further up or left, the better. The corresponding table displays the construction times and out-degrees of the bottom layer for each method. The values in the table are given as “mean $\pm$ standard deviation”.

10 dimensional uniform ball



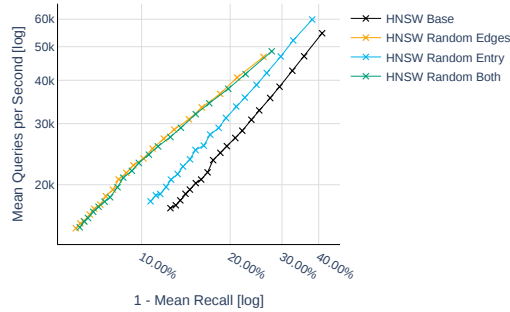
Method	Build time	Out-degree
HNSW Base	25.99 $\pm$ 0.09s	50.00 $\pm$ 0.00
HNSW Random Edges	25.85 $\pm$ 0.21s	53.00 $\pm$ 0.01
HNSW Random Entry	25.88 $\pm$ 0.21s	50.00 $\pm$ 0.00
HNSW Random Both	25.83 $\pm$ 0.18s	53.00 $\pm$ 0.01

50 dimensional uniform ball



Method	Build time	Out-degree
HNSW Base	76.20 $\pm$ 0.28s	50.00 $\pm$ 0.00
HNSW Random Edges	76.32 $\pm$ 0.43s	53.00 $\pm$ 0.01
HNSW Random Entry	76.31 $\pm$ 0.18s	50.00 $\pm$ 0.00
HNSW Random Both	76.17 $\pm$ 0.15s	53.00 $\pm$ 0.01

GooAQ sample



Method	Build time	Out-degree
HNSW Base	99.47 $\pm$ 4.27s	50.00 $\pm$ 0.00
HNSW Random Edges	99.33 $\pm$ 4.27s	53.00 $\pm$ 0.01
HNSW Random Entry	99.43 $\pm$ 4.44s	50.00 $\pm$ 0.00
HNSW Random Both	99.41 $\pm$ 4.45s	53.00 $\pm$ 0.01

Fig. 4. Comparison of single layer HNSW with and without random edges and random entry points. The curves show the QPS over error (1-recall) for each method. The further up or left, the better. The corresponding table displays the construction times and out-degrees of the bottom layer for each method. The values in the table are given as “mean $\pm$ standard deviation”.

by higher layers in default search, this heuristic is only useful for single-layer graph indices. We therefore limit this experiment to single-layer HNSW. The results are displayed in Fig. 4. On synthetic data, the heuristics do not diverge too much, with the random entry points performing slightly better than the static entry point. On GooAQ, the addition of random edges improves the recall significantly and the random entry points improve the recall slightly. The combination of both heuristics, does not further improve on the random edges. The random edges thus successfully add “shortcuts” to the graph, which reduce the average path length and improve the recall. While the performance of single-layer HNSW with random edges is not better than the standard HNSW (see Fig. 3), the random edges could be added to the bottom layer of a hierarchy as well, possibly improving the performance there too.